

Lab 1: Sequential vs Concurrent vs Parallel

1 Aim

To explore and understand the differences in execution time and performance between sequential, concurrent, and parallel approaches to solving a problem in Java.

2 Tasks

You are provided with three Java files (SequentialSum, SeqAndConcurrent, and ParallelSum) that sum the elements of an array in three ways: sequential, concurrent, and in parallel. The files are available on Brightspace. You need to compare the execution times of these three different way by summing a large array of integers and modifying the code to observe how changes affect performance.

2.1 Run and Compare

Run the sequential, concurrent, and parallel versions of the code several times. Observe and record the execution times for each approach. Analyse the differences in execution times between the three approaches.

2.2 Modify Array Size

Change the size of the array (e.g., increase from 100 million elements to 200 million). Observe how the execution times for each approach are affected by the array size.

```
/****Try changing the size of the array  
int arraySize = 100_000_000;  
int[] array = generateArray(arraySize);
```

Figure 1 arraySize variable. Can be found in the main function of both files: SeqAndConcurrentSum.java and ParallelSum.java.

2.3 Modify Number of Threads

In the concurrent version, change the number of threads (e.g., from 2 to 3 or even much more). Observe how this change impacts the performance and execution time. Consider how your system's number of CPU cores might influence these results.

```
/**** Try changing the number of threads */  
int numOfThreads = 2;  
runConcurrent(array, numOfThreads);
```

Figure 2 numOfThreads variable found in the main function of the SeqAndConcurrentSum.java file.

2.4 Adjust Threshold in Parallel Execution

In the parallel version, modify the THRESHOLD variable found in class SumTask, which determines the minimum chunk size for splitting tasks. Observe how different threshold values affect performance. Try to find the optimal threshold value for your system and array size.

```
/** try changing this value to see the impact  
private static final int THRESHOLD = 100_000;
```

Figure 3 Threshold variable in class SumTask in ParallelSum.java.